Angular Full Project Task

Covers everything from the basics up to Routing

Project Idea

"Store Manager" – a simple e-commerce product management app

Students will build a multi-page application that displays products fetched from an API, lets the user add a new product, view product details, and navigates between pages using routing.

> Note: Students may choose a different theme (e.g. Movies App, Students App, Recipes App...) as long as they meet all the technical requirements below.

Technical Requirements (all must be covered)

1. Project Structure & Modules

- Create the project using Angular CLI.
- Create at least one Feature Module (e.g. **ProductsModule**) and import it into **AppModule**.

2. Components

- At least 4 components, for example: **HomeComponent**, **ProductListComponent**, **ProductDetailsComponent**, **AddProductComponent**, **NavbarComponent**.
- Use Component Communication (**@Input** and **@Output**) between a parent and a child at least once (e.g. passing a product from the list to a card component).

3. Data Binding (all four types must be used)

- Interpolation **{{ }}**
- Property Binding **[ ]**
- Event Binding **( )**
- Two-way Binding **[(ngModel)]** (in the add-product form)

4. Directives

- ***ngFor** to display the list of products.
- ***ngIf** (e.g. show a "No products found" message when the list is empty, or a loading spinner).
- **[ngClass]** or **[ngStyle]** (e.g. highlight a product in a different color when its quantity is zero).

5. Pipes

- Use built-in pipes (**currency**, **date**, **uppercase**... depending on the data).
- Create at least one **Custom Pipe** (e.g. a pipe that truncates a long product description).

6. Services & Dependency Injection

- Create a **ProductService** responsible for all data operations (fetch, add, delete).
- The service must be injected into components via Dependency Injection, not instantiated manually with **new**.

7. HTTP Client (consuming an API)

- Use **HttpClientModule**.
- Fetch real data from a test API, such as **https://fakestoreapi.com/products** or **https://jsonplaceholder.typicode.com/posts**.
- Call **HttpClient** from inside the service (not directly inside the component).
- Handle the Loading state and errors (in case the request fails).

8. Routing (the main focus)

- Create a separate **AppRoutingModule**.
- At least 4 routes: **''** → Home, **'products'** → Product List, **'products/:id'** → Product Details (using **Route Parameters**), **'add-product'** → Add Product, **'**'** → Not Found page (Wildcard Route).
- Use **routerLink** in the navbar (not a plain **<a href="">**).
- Use **routerLinkActive** to highlight the current active link in the navbar.
- Use **ActivatedRoute** to read the **id** from the URL on the details page.
- Use the **Router** service to navigate programmatically after adding a product (e.g. redirect back to the products page after saving).

Suggested Page Structure

Page	Route	Purpose
Home	/	Simple welcome page
Products List	/products	Displays all products from the API in cards
Product Details	/products/:id	Shows details of a single product based on its id
Add Product	/add-product	Form to add a new product (two-way binding)
Not Found	any invalid URL	Simple 404 page

Grading Rubric

Criteria	Weight
Components + project structure	15%
Data Binding (all four types)	15%

Directives (ngFor, ngIf, ngClass)	10%
Pipes (built-in + custom)	10%
Service + Dependency Injection	15%
HTTP Client + Loading/Error handling	15%
Routing (routes, params, navigation)	20%
Total	**100%**

Submission

- Upload the code to a GitHub repository.
- Include a simple **README.md** with: project name, description, and setup steps (**npm install** then **ng serve**).

Tips for Students

- Start by setting up routing and empty components first, then fill in the content.
- Build the service first, before connecting it to components.
- Use **console.log** or DevTools to confirm the API data is correct before displaying it.